Site entraînement : Clients Commandes Produits

- ACTUEL
- TODO & DONE
  - 1️⃣ Initialisation du projet
  - 2️⃣ Modélisation du domaine
  - 3️⃣ Accès aux données (DAO / Repository)
  - 4️⃣ Couche service (logique métier)
  - 5️⃣ DTO et transformations
  - 6️⃣ Couche controller (API REST)
  - 7️⃣ Gestion des collections
  - 8️⃣ Gestion des cas limites
  - 9️⃣ Tests manuels
  - 🔟 Relecture “entretien”
- COMMANDES UTILES
- DONE

ACTUEL

- Initialiser pour pouvoir lancer le backend en Java pur

- [ ] Configuration Hibernate
  - [x] Création hibernate.cfg.xml
  - [x] Création classe HibernateUtils.java
  - [x] Appel d’Hibernate dans le main
  - [ ] Test
- [ ] **FIX PROBLEME CONNEXION BACKEND → DB**
  - [ ] Pourquoi le conteneur DB ne prend aucune mémoire

TODO & DONE

- Étape 1 : Initialisation du projet

  - [x] Configurer la base de données
  - [x] Vérifier que le script SQL est bien exécuté au démarrage
  - [x] Définir l’arborescence (controller / service / repository / model / dto)
- Étape 2 : Modélisation du domaine

  - [ ] Créer les entités correspondant aux tables
  - [ ] Associer chaque entité à sa table
  - [ ] Mettre en place les relations entre entités
  - [ ] Vérifier la cohérence avec le schéma SQL
- Étape 3 : Accès aux données (Repository)

  - [ ] Créer une interface par entité
  - [ ] Exposer les opérations CRUD de base
  - [ ] Tester la récupération des données existantes
  - [ ] Gérer l’absence de résultat
- Étape 4 : Couche service (logique métier)

1️⃣ Initialisation du projet

- Créer un projet backend Java
- Définir l'arborescence (controller / service / repository / model / dto)
- Configurer la base de données
- Vérifier que le script SQL est bien exécuté au démarrage

2️⃣ Modélisation du domaine

- Créer les entités correspondant aux tables
- Associer chaque entité à sa table
- Mettre en place les relations entre entités
- Vérifier la cohérence avec le schéma SQL

3️⃣ Accès aux données (DAO / Repository)

- Créer une interface par entité
- Exposer les opérations CRUD de base
- Tester la récupération des données existantes
- Gérer l'absence de résultat

4️⃣ Couche service (logique métier)

- Créer un service par domaine métier
- Centraliser la logique applicative
- Appeler les repositories depuis les services
- Gérer les cas d'erreur fonctionnels
- Définir les transactions

5️⃣ DTO et transformations

- Définir les objets échangés avec l'extérieur
- Ne jamais exposer les entités directement
- Transformer entités → DTO
- Transformer DTO → entités

6️⃣ Couche controller (API REST)

- Exposer les endpoints HTTP
- Respecter la sémantique HTTP
- Valider les entrées
- Appeler uniquement les services
- Retourner des réponses propres

7️⃣ Gestion des collections

- Manipuler les listes de résultats
- Choisir List ou Set selon le besoin
- Utiliser Map si un accès clé → valeur est pertinent
- Vérifier le comportement avec les relations

8️⃣ Gestion des cas limites

- Ressource inexistante
- Données invalides
- Relations absentes
- Suppression avec dépendances

9️⃣ Tests manuels

- Tester chaque endpoint
- Vérifier la cohérence des données
- Tester les erreurs attendues
- Observer les transactions

🔟 Relecture “entretien”

- Être capable d’expliquer chaque couche
- Justifier les choix techniques
- Expliquer le rôle d’Hibernate
- Décrire le cycle d’une requête HTTP
- Savoir ce qui se passe en base

COMMANDES UTILES

Lancer le projet

```
docker-compose up -d
```

Lancer le backend (mais le docker-compose up lance déjà tout)

```
cd backend
mvn clean install
cd target
java -jar app.jar OU java -jar .\demo-1.0-SNAPSHOT.jar
```

Entrer dans la DB

```
docker exec -it vitrine-postgres psql -U postgres -d vitrine-db
```

DONE

- [x] Initialiser backend avec Maven
- [x] Ajout des dépendances dans le pom.xml (Hibernate, JPA, JDBC Driver, Json...)
- [ ] Configuration Hibernate
  - [x] Création hibernate.cfg.xml
  - [x] Création classe HibernateUtils.java
  - [x] Appel d’Hibernate dans le main
  - [ ] Test

Lundi 23/02/2026

- [x] Configurations Docker (lancement du backend automatique)
  - [x] Ajout dépendances pom.xml
  - [x] Modifications hibernate.cfg.xml
  - [x] Ajout Dockerfile au backend
- [x] **FIX PROBLEME CONNEXION BACKEND → DB**
  - [x] Mettre en place un serveur http (dans le Main.java, pour faire tourner en boucle le serveur)
  - [x] Controller, service & repo créés pour “Hello” (en guise d’exemple)